TMM4128:
Machine Learning for Engineers

Andrei Lobov
e-mail: andrei.lobov@ntnu.no
phone: +47 45 913 455
room: 213, Verkstedteknisk
web: http://www.lobov.biz/academia

Slides:> http://lobov.biz/academia/mle/250206

Outline : Ensemble Learning and Random Forests

- Definition
- Voting Classifiers
- Bagging and Pasting
  - Bagging and Pasting in Scikit-Learn
  - Out-of-Bag Evaluation
- Random Patches and Random Subspaces
- Random Forests
  - Extra-Trees
  - Feature Importance
- Boosting
  - AdaBoost
  - Gradient Boosting
- Stacking
- Examples

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019 (Available online) / **Chapter 7**

Definition

A group of predictors is called an *ensemble*.

Getting better predictions than individual predictors. Combine good predictors into an even better predictor.

Ensemble methods examples: bagging, boosting, stacking and Random Forests.

Voting Classifiers

Steps 1-9

Voting classifiers

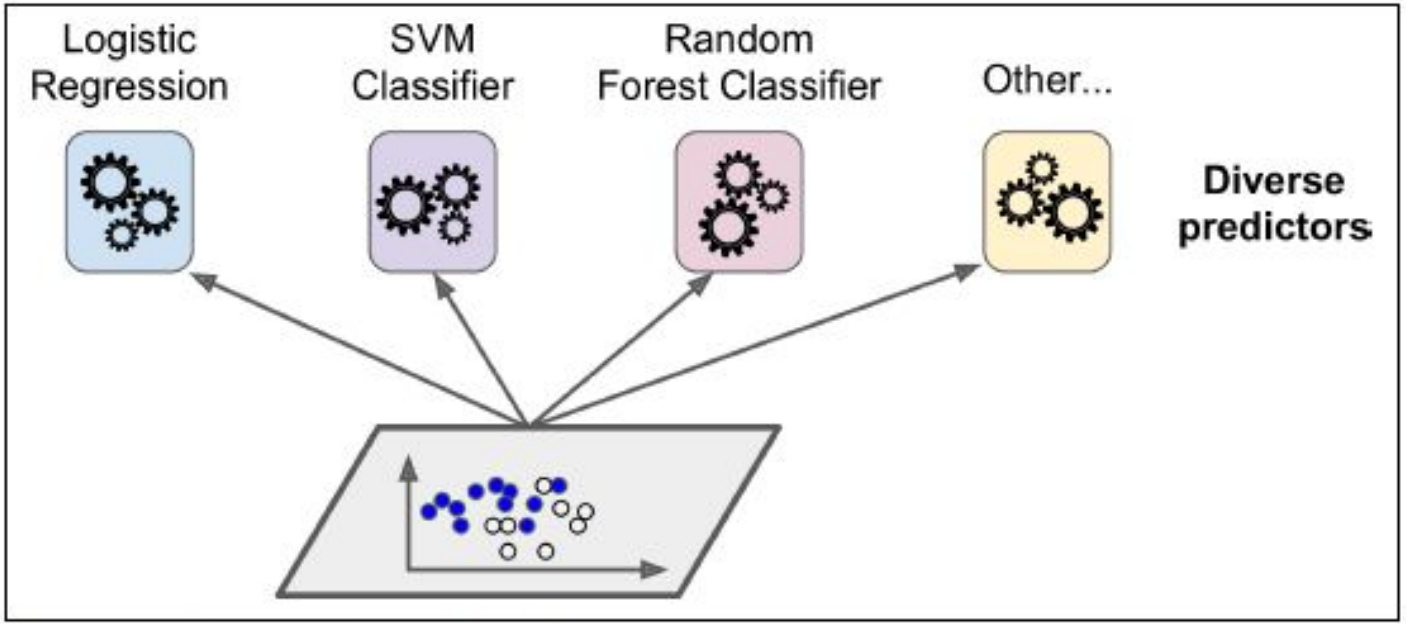


Figure 7-1. Training diverse classifiers

Voting classifiers: majority-vote classifier

Achieves a higher accuracy than the best classifier in the ensemble.

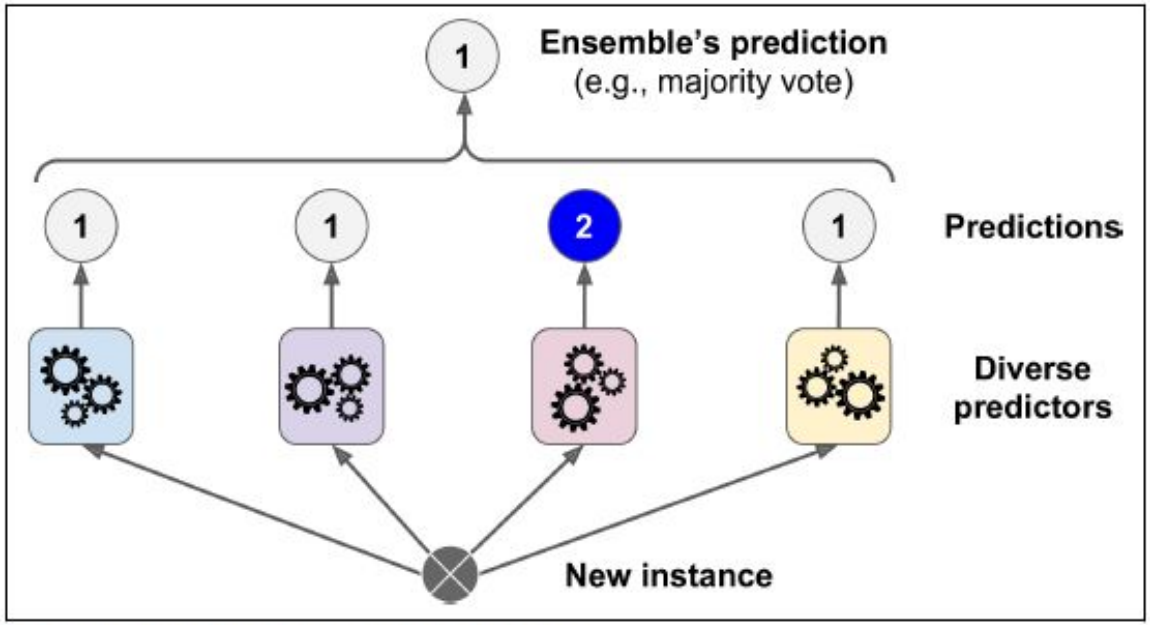


Figure 7-2. Hard voting classifier predictions

From “weak learners” to “strong learner” through the ensemble

The ensemble of many weak learners (i.e. slightly better than random guessing) can still be a strong learner (achieving high accuracy). Having a sufficient number of weak learners and they are sufficiently diverse.

The law of large numbers

$$P_n^k = C_n^k p^k q^{n-k}$$

$$q = 1 - p$$

Obtaining **majority of heads** probability after 1000 tosses ~ 75%
After 10000 tosses ~ 97%

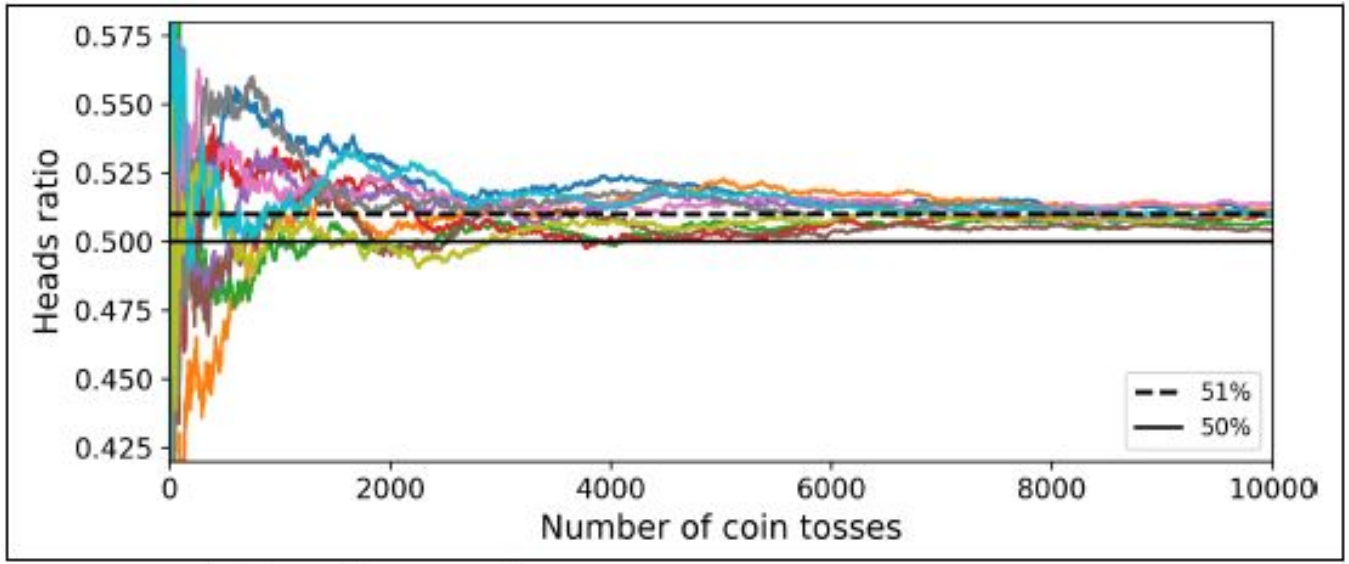


Figure 7-3. The law of large numbers

Voting classifiers : VotingClassifier

VotingClassifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression()
rnd_clf = RandomForestClassifier()
svm_clf = SVC()

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='hard')
voting_clf.fit(X_train, y_train)
```

Voting classifiers : VotingClassifier : hard / soft

VotingClassifier

voting : *{'hard', 'soft'}, default='hard'*

If 'hard', uses predicted class labels for majority rule voting. Else if 'soft', predicts the class label based on the argmax of the sums of the predicted probabilities, which is recommended for an ensemble of well-calibrated classifiers.

weights : *array-like of shape (n_classifiers,), default=None*

Sequence of weights (`float` or `int`) to weight the occurrences of predicted class labels (`hard` voting) or class probabilities before averaging (`soft` voting). Uses uniform weights if `None`.

Soft voting: to predict the class with the highest class probability, averaged over all the individual classifiers. It often achieves higher performance than hard voting because it gives more weight to highly confident votes.

Classifiers vs Ensemble

```
>>> from sklearn.metrics import accuracy_score
>>> for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
...     clf.fit(X_train, y_train)
...     y_pred = clf.predict(X_test)
...     print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
...
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.888
VotingClassifier 0.904
```

hard

```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.92
```

soft

Bagging and Pasting

Steps 10-17

Bagging and pasting (1)

One way - use very different classifiers. Another (here): train several predictors on different random samples of the training set.

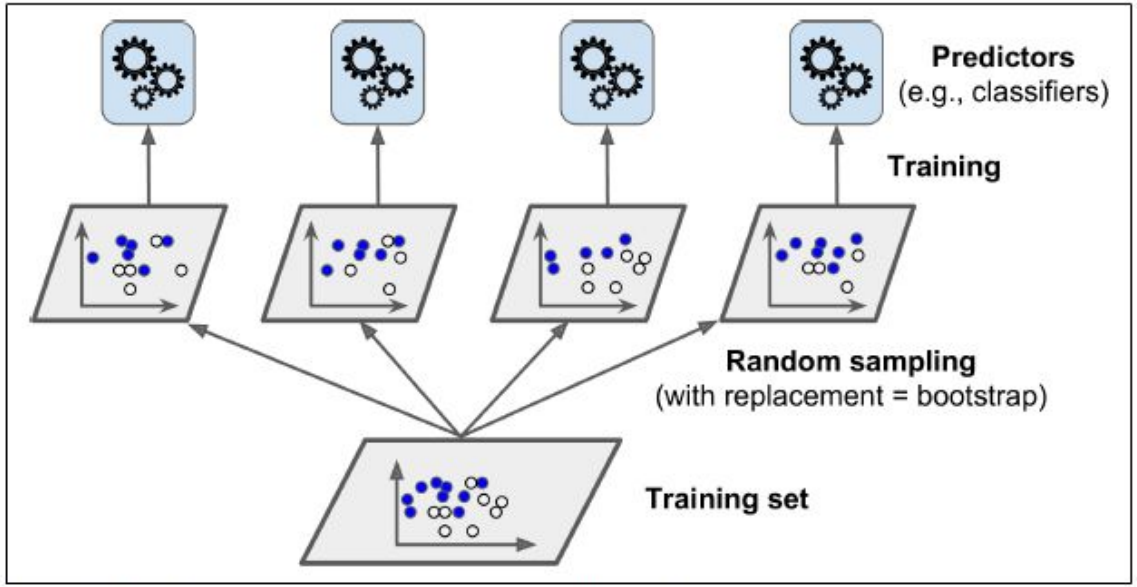


Figure 7-4. Pasting/bagging training set sampling and training

Bagging and pasting (2)

Same training algorithm for every predictor, training predictors on different **random subsets of the training set**.

Bagging : bootstrap aggregating - sampling *with* replacement.

Pasting : sampling is performed *without* replacement.

Both bagging and pasting allow training instances to be sampled several times across multiple predictors, but only bagging allows training instances to be sampled several times for the same predictor.

Bagging and pasting (3)

Can scale well as predictors can be trained in parallel on different CPUs or servers.

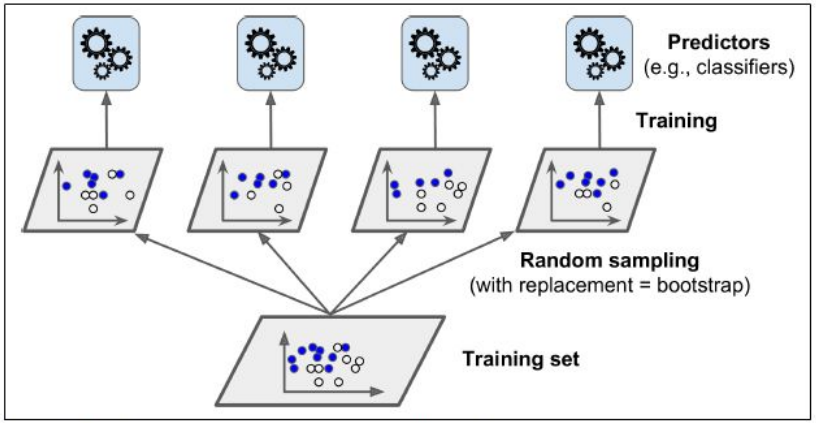


Figure 7-4. Pasting/bagging training set sampling and training

Bagging and Pasting in Scikit-Learn (1)

BaggingClassifier

BaggingRegressor

n_jobs : number of CPU cores to use for training. (-1 - use all available).

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

bag_clf = BaggingClassifier(
    DecisionTreeClassifier(), n_estimators=500,
    max_samples=100, bootstrap=True, n_jobs=-1)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

Bagging and Pasting in Scikit-Learn (2)

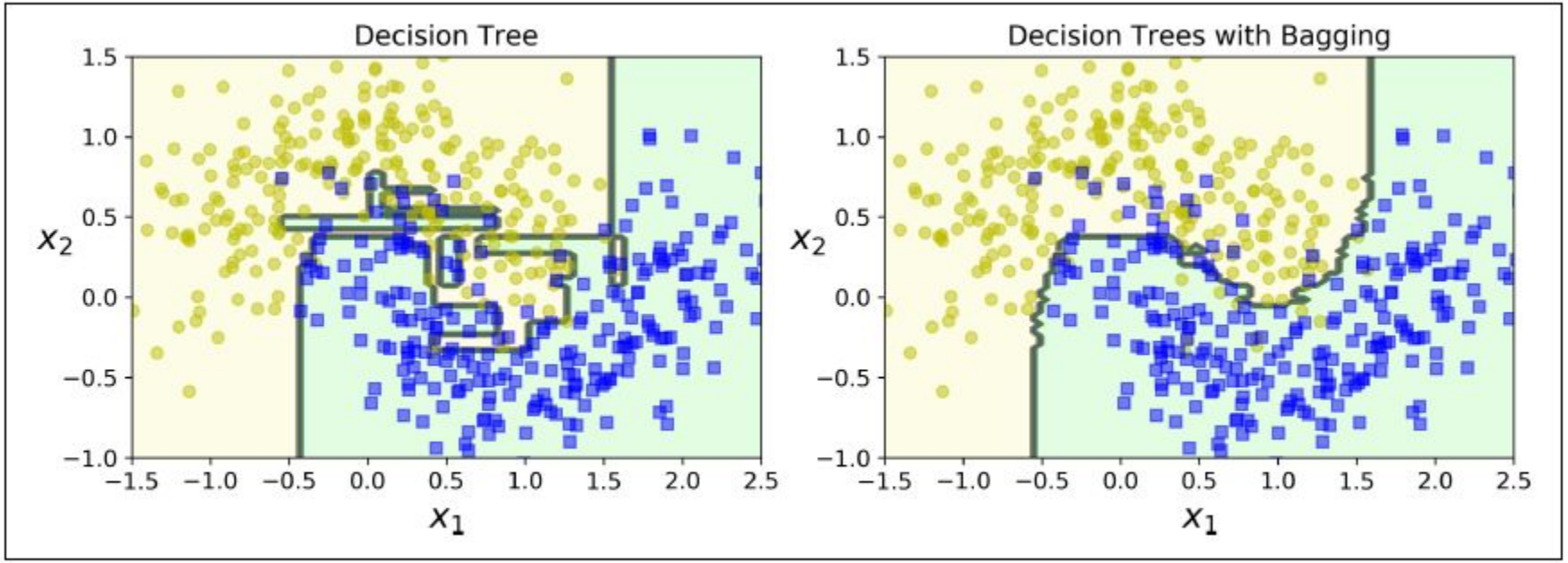


Figure 7-5. A single Decision Tree versus a bagging ensemble of 500 trees

Out-of-Bag Evaluation (1)

With bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all.

By default a BaggingClassifier samples m training instances with replacement (bootstrap=True), where m is the size of the training set.

About 63% of the training instances are sampled on average for each predictor.

The remaining 37% of the training instances that are not sampled are called **out-of-bag** (oob) instances. Note that they are not the same 37% for all predictors.

Out-of-Bag Evaluation (2)

Since a predictor never sees the oob instances during training, it can be evaluated on these instances, without the need for a separate validation set.

Evaluation score is available through the **oob_score_**

```
>>> bag_clf = BaggingClassifier(
...     DecisionTreeClassifier(), n_estimators=500,
...     bootstrap=True, n_jobs=-1, oob_score=True)
...
>>> bag_clf.fit(X_train, y_train)
>>> bag_clf.oob_score_
0.90133333333333332
```

close enough!:

```
>>> from sklearn.metrics import accuracy_score
>>> y_pred = bag_clf.predict(X_test)
>>> accuracy_score(y_test, y_pred)
0.91200000000000003
```

Out-of-Bag Evaluation : class probabilities for each training instance

68.25% probability of the first training instance to belong to the positive class (31,75% of belonging to the negative class).

```
>>> bag_clf.oob_decision_function_
array([[0.31746032, 0.68253968],
       [0.34117647, 0.65882353],
       [1.        , 0.        ],
       ...
       [1.        , 0.        ],
       [0.03108808, 0.96891192],
       [0.57291667, 0.42708333]])
```

Random Patches and Random Subspaces

Random Patches and Random Subspaces

"Focus" on features rather than training instances: a predictor will be trained on a **random subset of the input features**.

BaggingClassifier : **max_features** and **bootstrap_features** hyperparameters.'

Particularly useful when with high-dimensional inputs (such as images).

Sampling both training instances and features is called the **Random Patches method**.
Keeping all training instances (i.e., **bootstrap**=False and **max_samples**=1.0) but sampling features (i.e., **bootstrap_features**=True and/or **max_features** smaller than 1.0) is called the **Random Subspaces method**.

Random Forests

Steps 18-28

Random Forests (1)

Random Forest is an **ensemble of Decision Trees**

It is generally trained via the bagging method (or sometimes pasting), typically with max_samples set to the size of the training set.

RandomForestClassifier.

RandomForestRegressor.

Random Forests (2)

The Random Forest introduces extra randomness when growing trees.

Instead of searching for the very best feature when splitting a node, it searches for the best feature among **a random subset of features**.

The algorithm results in greater tree diversity, which trades a higher bias for a lower variance. → Generally yields an overall better model.

Extra-Trees

When a Random Forest is grown, at each node only a random subset of the features is considered for splitting.

It is possible to make trees even more random by also using random thresholds for each feature rather than searching for the best possible thresholds (like regular Decision Trees do).

ExtraTreesClassifier
ExtraTreesRegressor

Feature importance (1)

- Random Forests make it easy to measure the relative **importance of each feature**.
  - how much the tree nodes that use that feature reduce impurity on average (across all trees in the forest).
  - quick understanding of what features actually matter, in particular if you need to perform feature selection.

```
>>> from sklearn.datasets import load_iris
>>> iris = load_iris()
>>> rnd_clf = RandomForestClassifier(n_estimators=500, n_jobs=-1)
>>> rnd_clf.fit(iris["data"], iris["target"])
>>> for name, score in zip(iris["feature_names"], rnd_clf.feature_importances_):
...     print(name, score)
...
sepal length (cm) 0.112492250999
sepal width (cm) 0.0231192882825
petal length (cm) 0.441030464364
petal width (cm) 0.423357996355
```

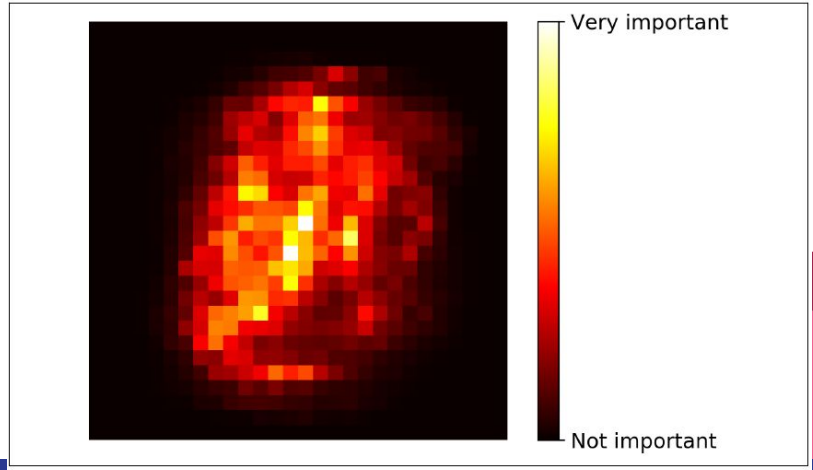


Figure 7-6. MNIST pixel importance (according to a Random Forest classifier)

Boosting

AdaBoost
Gradient boosting
Steps 29-54

Boosting

Boosting (originally called *hypothesis boosting*) refers to **any Ensemble method that can combine several weak learners into a strong learner**.

Most boosting methods is to train predictors sequentially, each trying to correct its predecessor.

The most popular are *AdaBoost* (short for Adaptive Boosting) and *Gradient Boosting*.

AdaBoost (1)

One way for a new predictor to correct its predecessor is to pay a bit more attention to the training instances that the predecessor underfitted. This results in new predictors focusing more and more on the hard cases.

AdaBoost classifier: a first base classifier (such as a Decision Tree) is trained and used to make predictions on the training set. The **relative weight of misclassified training instances is then increased**. A second classifier is trained using the updated weights and again it makes predictions on the training set, weights are updated, and so on

AdaBoost (2)

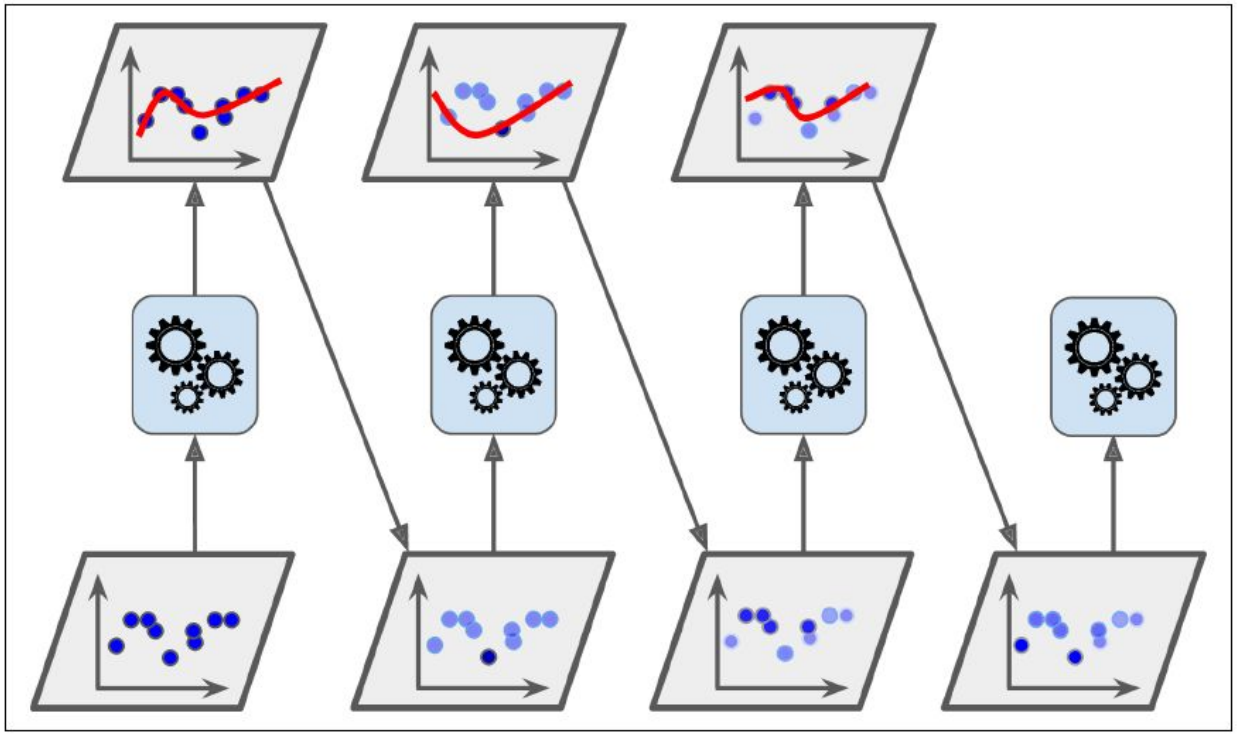


Figure 7-7. AdaBoost sequential training with instance weight updates

AdaBoost: Looking at decision boundaries

Decision boundaries of five consecutive predictors on the moons dataset (in this example, each predictor is a highly regularized SVM classifier with an RBF kernel).

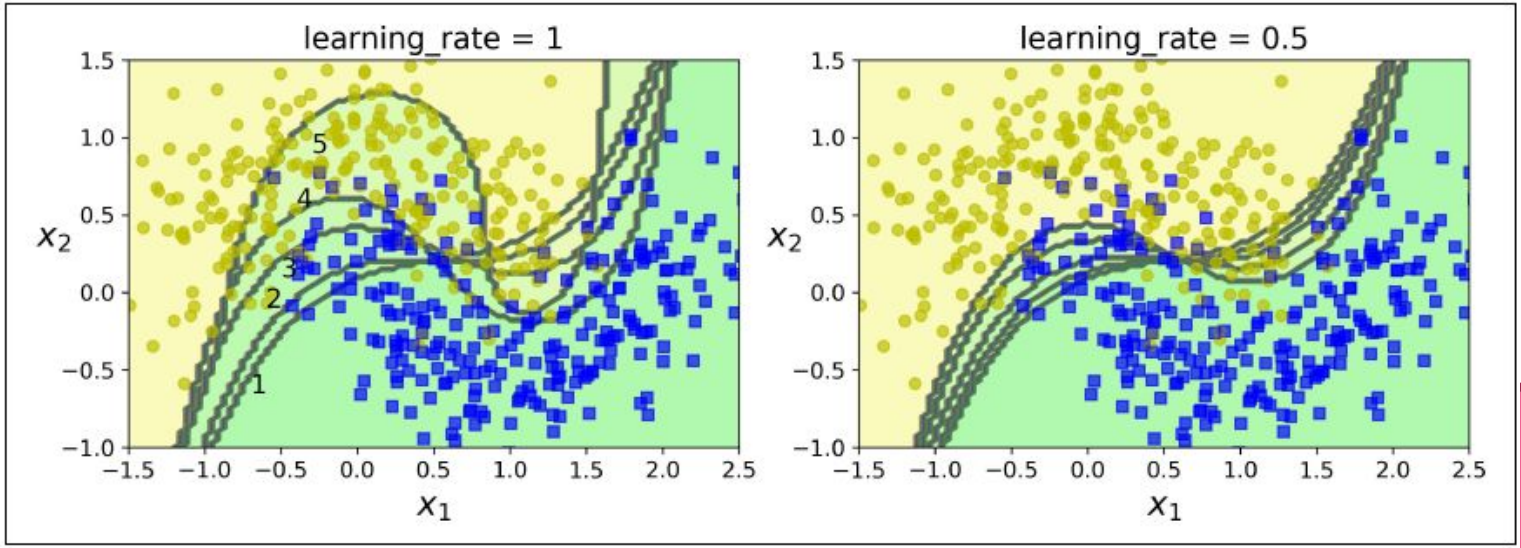


Figure 7-8. Decision boundaries of consecutive predictors

AdaBoost algorithm (1)

1. Each instance weight $w^{(i)}$ is initially set to $1/m$. A first predictor is trained and its weighted error rate r_1 is computed on the training set.

Equation 7-1. Weighted error rate of the j^{th} predictor

$$r_j = \frac{\sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)}}{\sum_{i=1}^{m} w^{(i)}}$$

where $\hat{y}_j^{(i)}$ is the j^{th} predictor's prediction for the i^{th} instance.

AdaBoost algorithm (2)

2. The predictor's weight α_j is then computed using Equation 7-2, where η is the learning rate hyperparameter (defaults to 1). The more accurate the predictor is, the higher its weight will be. If it is just guessing randomly, then its weight will be close to zero. However, if it is most often wrong (i.e., less accurate than random guessing), then its weight will be negative.

Equation 7-2. Predictor weight

$$\alpha_j = \eta \log \frac{1 - r_j}{r_j}$$

AdaBoost algorithm (3)

3. Next the instance weights are updated using Equation 7-3: the misclassified instances are boosted.

Equation 7-3. Weight update rule

$$\text{for } i = 1, 2, \cdots, m$$

$$w^{(i)} \leftarrow \begin{cases} w^{(i)} & \text{if } \widehat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp\left(\alpha_j\right) & \text{if } \widehat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

4. Then all the instance weights are normalized (i.e., divided by $\sum_{i=1}^{m} w^{(i)}$).

AdaBoost algorithm (4)

5. Finally, a new predictor is trained using the updated weights, and the whole process is repeated (the new predictor's weight is computed, the instance weights are updated, then another predictor is trained, and so on). The algorithm stops when the desired number of predictors is reached, or when a perfect predictor is found.

AdaBoost : Marking predictions (1)

To make predictions, AdaBoost simply computes the predictions of all the predictors and weighs them using the predictor weights α_j. The predicted class is the one that receives the majority of weighted votes

Equation 7-4. AdaBoost predictions

$$\hat{y}(\mathbf{x}) = \underset{k}{\operatorname{argmax}} \sum_{\substack{j=1 \\ \hat{y}_j(\mathbf{x}) = k}}^{N} \alpha_j \quad \text{where } N \text{ is the number of predictors.}$$

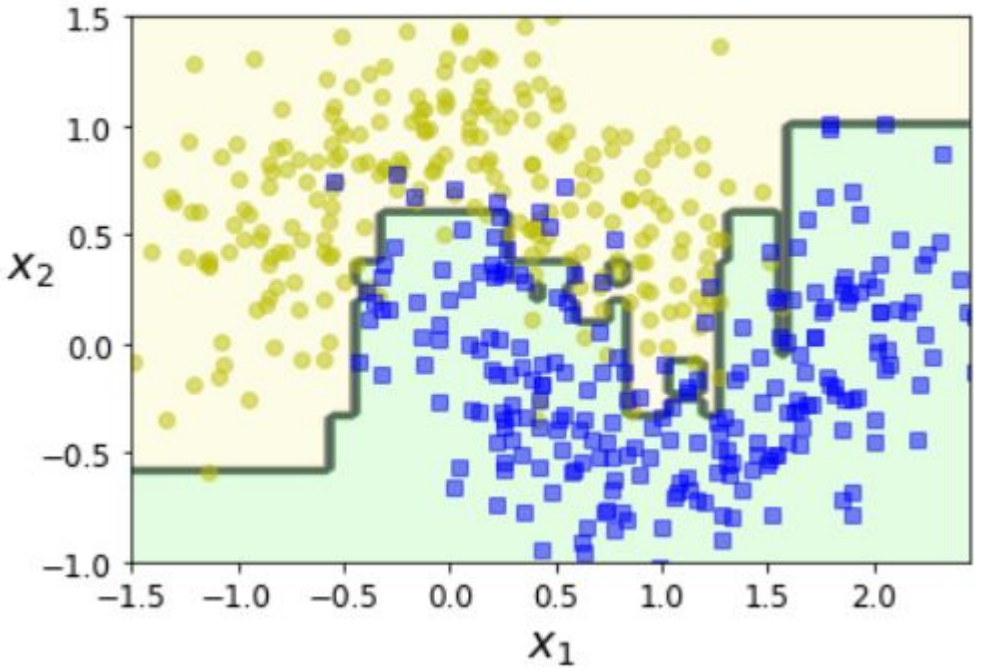
AdaBoost : Marking predictions (2)

AdaBoostClassifier

```
from sklearn.ensemble import AdaBoostClassifier

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=1), n_estimators=200,
    algorithm="SAMME.R", learning_rate=0.5)
ada_clf.fit(X_train, y_train)
```

```
plot_decision_boundary(ada_clf, X, y)
```



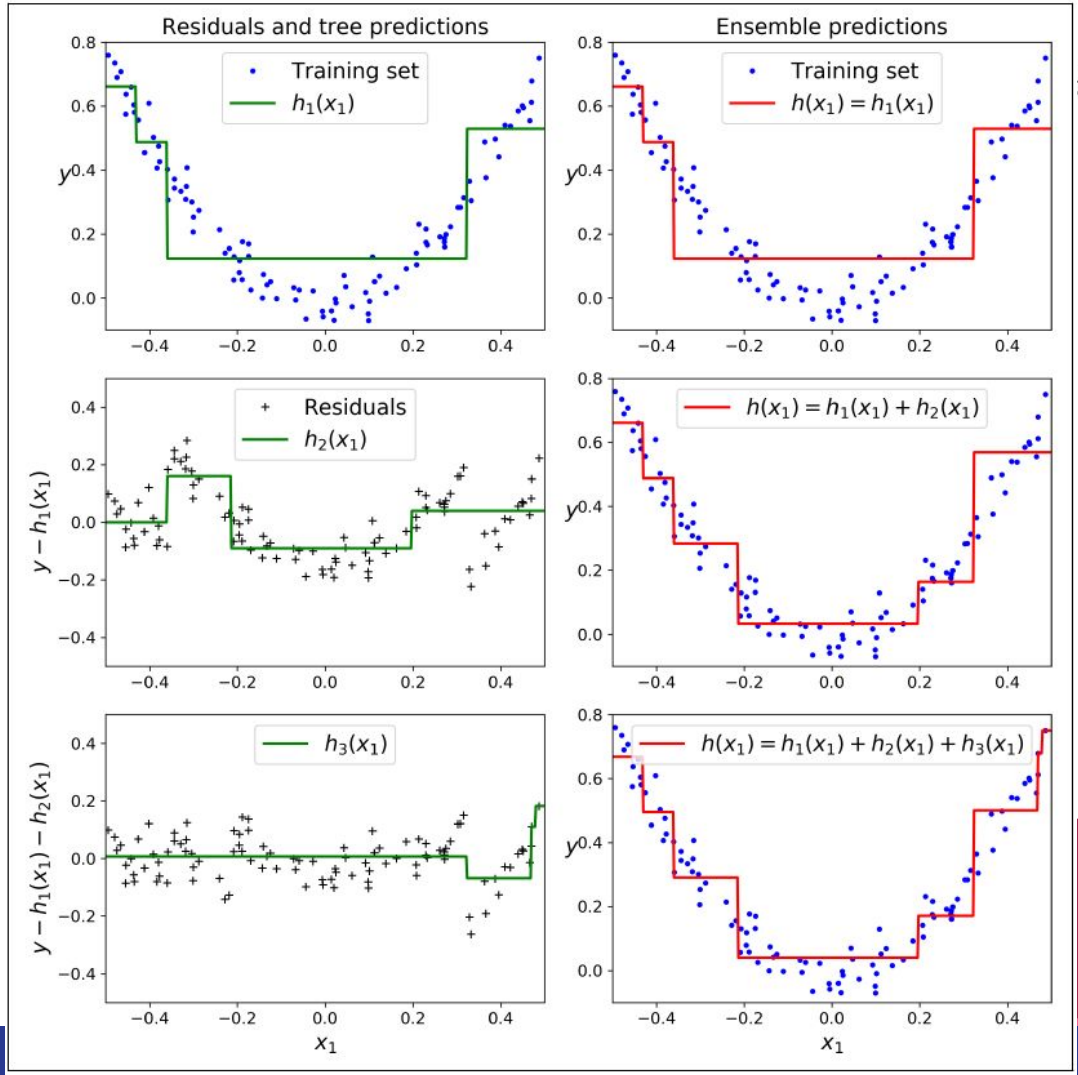
Gradient boosting

Just like AdaBoost, Gradient Boosting works by sequentially adding predictors to an ensemble, each one correcting its predecessor. However, instead of tweaking the instance weights at every iteration like AdaBoost does, this method tries to fit the new predictor to the *residual errors* made by the previous predictor.

GradientBoostingRegressor

GradientBoostingClassifier

Gradient Boosting : Example



Gradient Boosting Regression Trees (GBRT): underfitting and overfitting

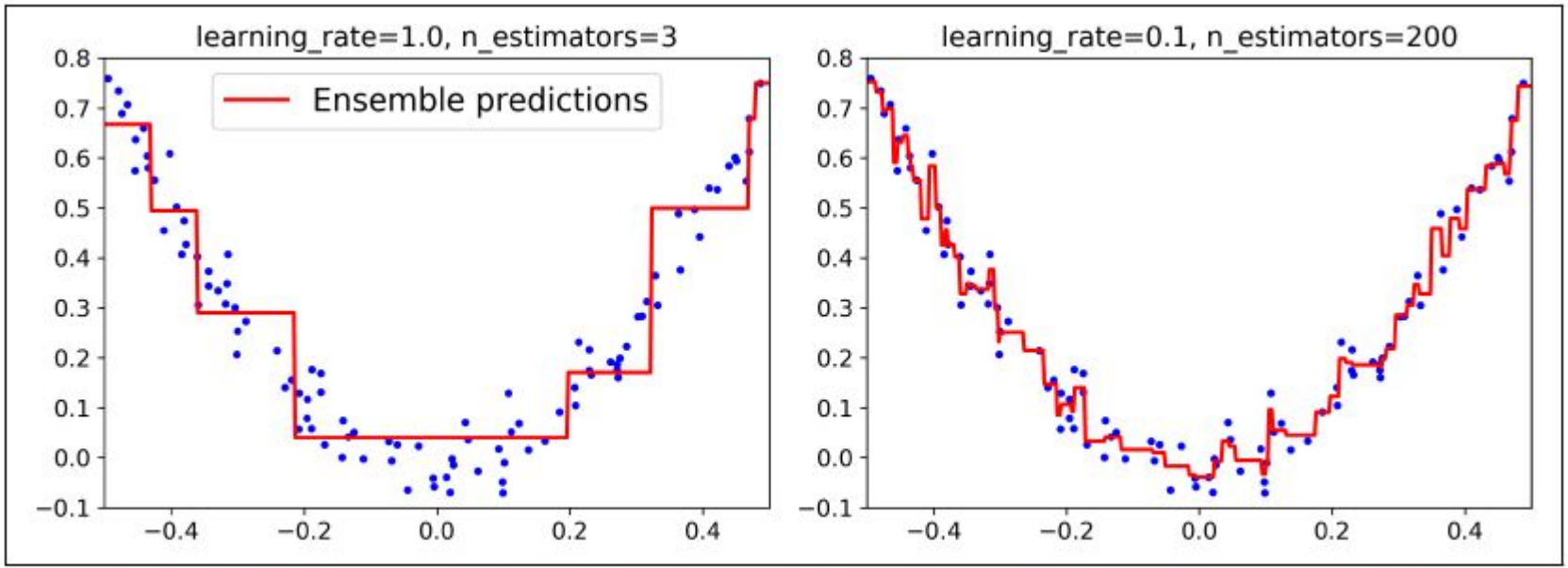


Figure 7-10. GBRT ensembles with not enough predictors (left) and too many (right)

Tuning the number of trees

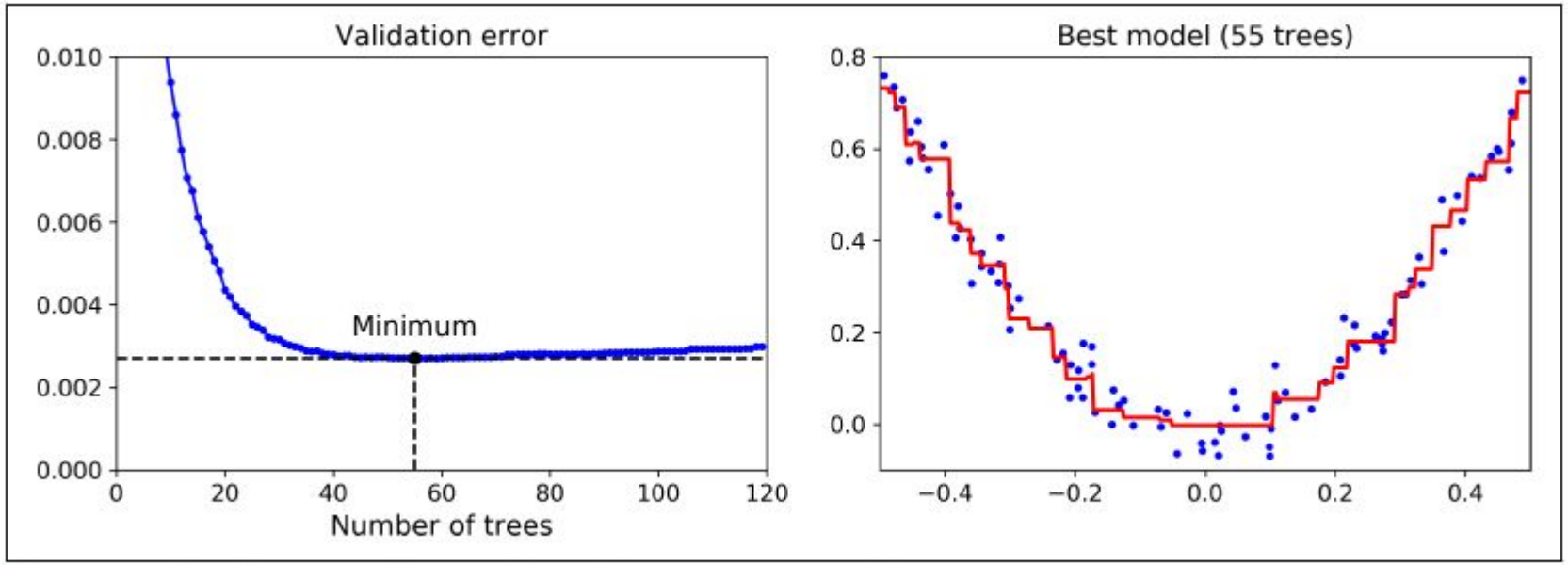


Figure 7-11. Tuning the number of trees using early stopping

Stacking

Stacking

Stacking (short for [stacked generalization]): Instead of using trivial functions (such as hard voting) to aggregate the predictions of all predictors in an ensemble, to train a model to perform this aggregation.

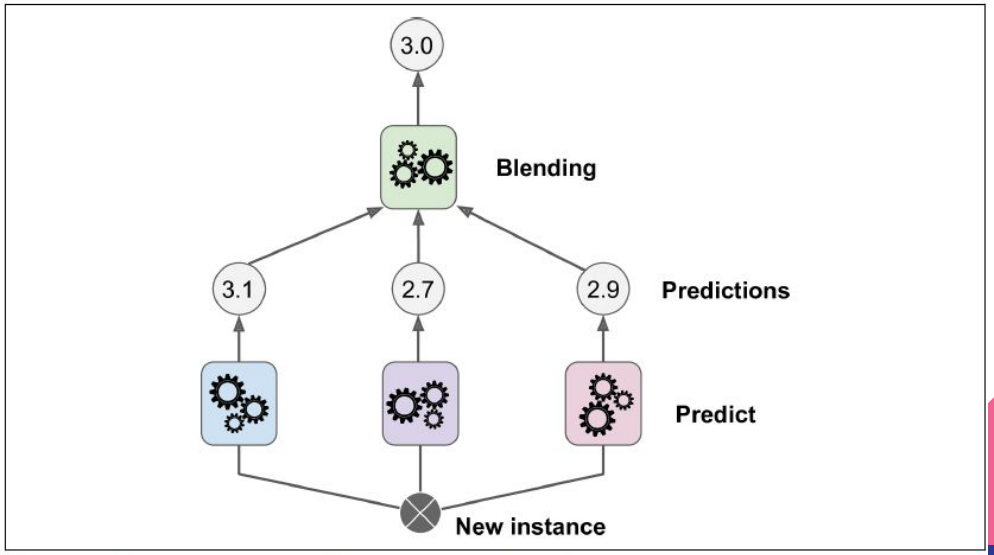


Figure 7-12. Aggregating predictions using a blending predictor

Training the first layer

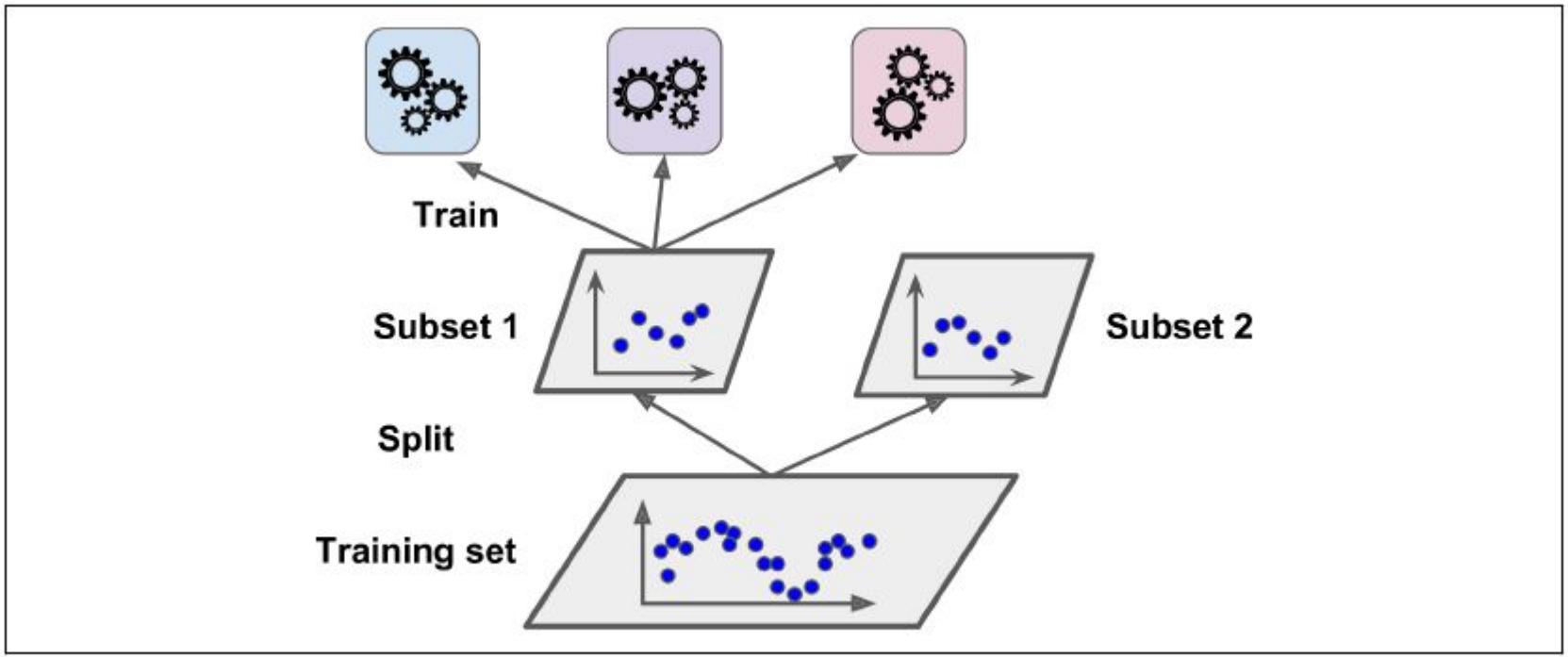


Figure 7-13. Training the first layer

Training the blender

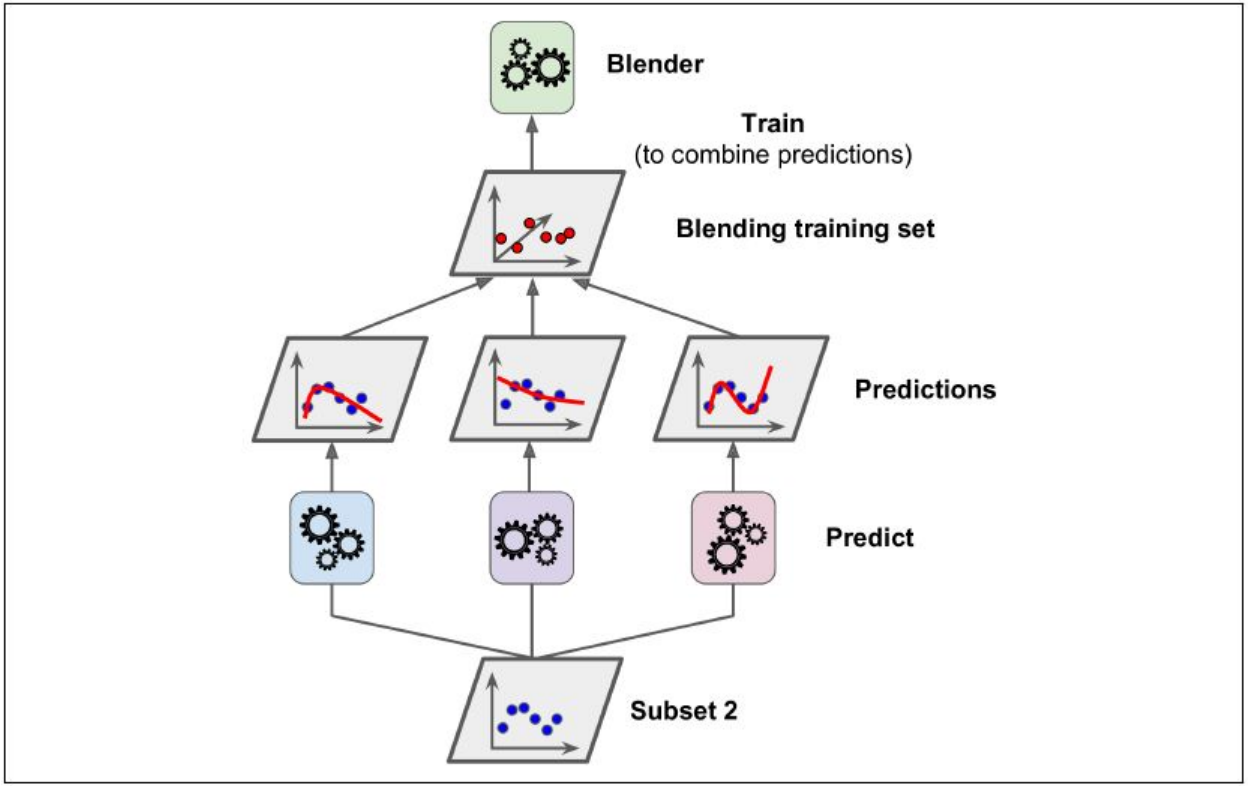


Figure 7-14. Training the blender

Making predictions

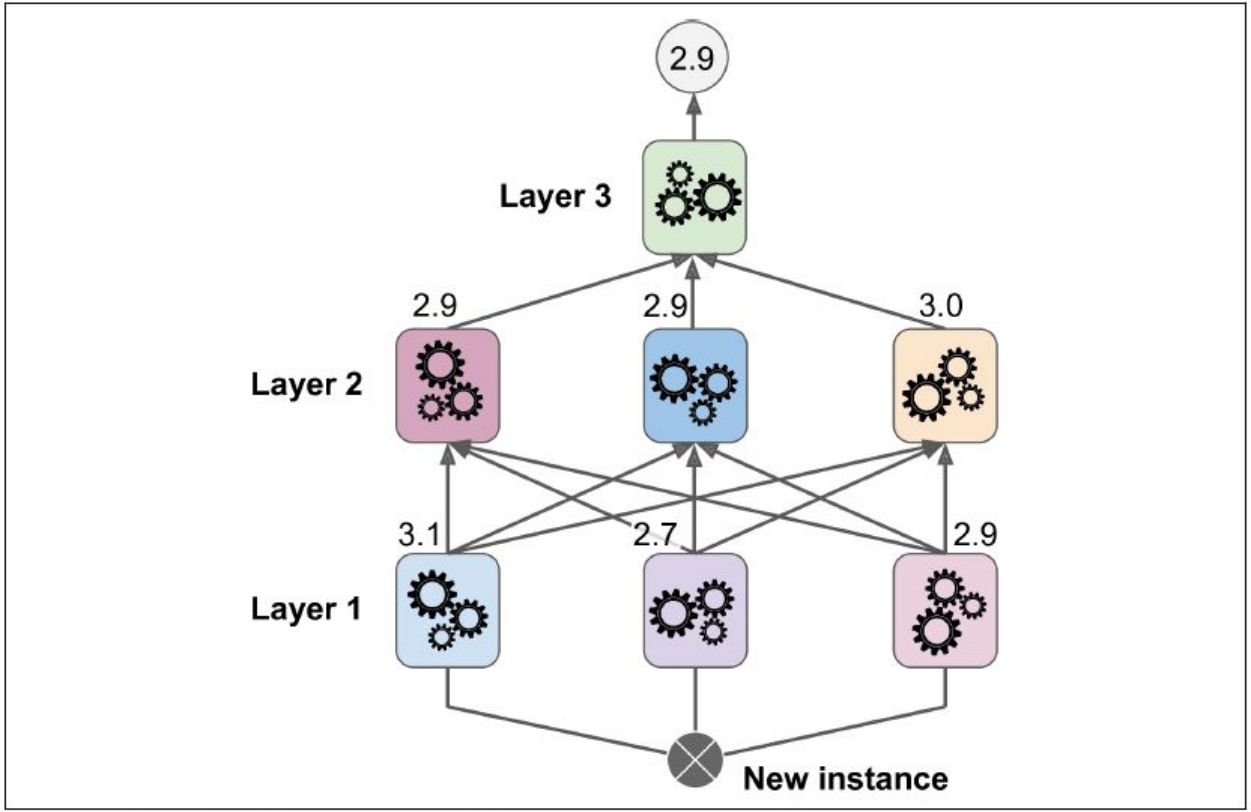


Figure 7-15. Predictions in a multilayer stacking ensemble

Examples

Yue Wang, Daniel Y. Mo, Mitchell M. Tseng, Mapping customer needs to design parameters in the front end of product design by applying deep learning, CIRP Annals, Volume 67, Issue 1, 2018, Pages 145-148, ISSN 0007-8506

Rapaccini, M., Cadonna, V. L., Leoni, L., & De Carlo, F. (2022). Application of machine learning techniques for cost estimation of engineer to order products. International Journal of Production Research, 61(20), 6978–7000.

Florian Klocker, Reinhard Bernsteiner, Christian Ploder, Martin Nocker. A Machine Learning Approach for Automated Cost Estimation of Plastic Injection Molding Parts, Cloud Computing and Data Science, 2023;4(2):87-111.

Takeaways

- Combine (in parallel and/or sequentially) good predictors into an even better predictor - an Ensemble.
- Evaluating feature importance.
- Boosting individuals and features.